

PERIYAR UNIVERSITY

Vidhyaa Arts and Science College

COLLEGE CODE: PER 177

PocketSmart AI

Smart Budget & Recommendation Assistant

Gemini Powered · FastAPI · Jinja2

TEAM LEADER	Gokulnathan J NM ID: 7D1B52F470EB3B3BAFA4FCAF9C0DC8B8 Email: gokulnathangokulnathan2006@gmail.com
TEAM MEMBERS	Giridharan Raja Kubenthiran T Santhosh V Thamaraiselvan S

Table of Contents

Phase 1: Brainstorming & Ideation Phase

Phase 2: Requirement Analysis Phase

Phase 3: Project Design Phase

Phase 4: Project Planning Phase

Phase 5: Project Development Phase

Phase 6: Project Testing Phase

Phase 7: Project Documentation Phase

Phase 8: Project Demonstration Phase

Technology Stack

Python · FastAPI · Uvicorn · Jinja2 · HTML & CSS · Google Gemini 1.5 Flash Pro · JWT Authentication

1. Brainstorming & Ideation Phase

Project Overview

PocketSmart AI is a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services. It analyzes user preferences, budgets and contextual needs using Google Gemini 1.5 Flash Pro and generates curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO. It is built with FastAPI for the backend and an HTML, CSS and Jinja2 frontend, and it turns budgeting into an intelligent, user-friendly experience through smart planners, adaptive forms and a recommendation engine.

Problem Statement

Managing budgets across different life needs, such as home decor, event planning or jewelry shopping, can be overwhelming because of the wide variety of products, platforms and price ranges. Users have to compare many websites by hand, and it is hard to stay within a budget while still getting good style, quality and functionality. There is a need for one tool that understands the budget and the context and suggests the right options across platforms.

Proposed Idea

Build a web application where the user picks a planner, enters a budget and a few preferences, and receives AI-generated recommendations instantly. PocketSmart AI offers three planners in one interface: the Home Budget Planner, the Party Planner and the Jewelry Budget Planner. Registered users also get a dashboard and a history of past recommendations.

Core Scenarios Identified

- **Home Interior Planning** – the user enters a budget, chooses room types (Living Room, Kitchen, Bedroom) and quantities of lights, ceiling fans and dining tables, and gets cost-effective options from platforms like IKEA and Amazon, balanced across function, style and price.
- **AI-Based Party Budget Planning** – the user enters the total budget, guest count, event type and venue details. The AI splits the budget across catering, decoration and entertainment, with options from Swiggy, Zomato and OYO, tailored to the event (birthday, corporate, wedding).
- **Jewelry Recommendations for Occasions** – the user enters a budget, occasion and style preferences, and can upload an outfit image. Gemini considers colour coordination and aesthetics and suggests matching jewelry from Amazon and Flipkart.

Target Users

- Individuals and families decorating or furnishing their homes on a fixed budget.
- People organizing birthdays, corporate events or weddings without financial guesswork.
- Shoppers looking for jewelry that matches their outfit, occasion and budget.
- E-commerce and lifestyle platforms and developers who want an AI recommendation layer.

Expected Benefit

A single interface that saves time and effort, keeps spending within budget and gives personalized, context-aware suggestions across multiple trusted platforms for home, party and jewelry needs.

2. Requirement Analysis Phase

Functional Requirements

- Provide a home page with links to the three planner modules and user actions.
- Provide planner forms for Home (budget, rooms, quantities), Party (budget, guests, event type, venue) and Jewelry (budget, occasion, style, optional outfit image).
- Generate budget-aware recommendations from Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO and similar platforms using Gemini 1.5 Flash Pro.
- Analyze an uploaded outfit image together with text input (multimodal) for jewelry matching.
- Support registration, login, logout and JWT token-based access to protected routes.
- Store session data and show a dashboard and recommendation history for each user.
- Display the results in clean, card-like layouts with product names, details and prices.

Non-Functional Requirements

- Lightweight, responsive interface built with HTML, CSS and Jinja2 templates.
- Modular backend with separate routes, services and models, so features can be upgraded easily.
- Gemini API key stored in a .env file and never shared in the code.
- Secure authentication and session handling; CORS configured for frontend communication.
- Input validation and fallback recommendations when the AI returns insufficient results.
- Graceful error handling with clear messages.

Inputs and Outputs

Feature	Input	Output
Home Planner	Budget, room types, item quantities	Cost-effective furniture, decor and lighting options per category
Party Planner	Budget, guest count, event type, venue details	Budget split across catering, decoration, entertainment and venue options
Jewelry Planner	Budget, occasion, style, optional outfit image	Style-matched jewelry options from shopping platforms
History	User session	Past queries and recommendations for review or re-use

Pre-requisites

Requirement	Details
Python 3.10+	Install from the official site, tick "Add Python to PATH", verify with <code>python --version</code>
FastAPI	Backend framework (official docs and tutorials available)
Uvicorn	ASGI server; installed with <code>pip install uvicorn</code>
Jinja2	HTML templating engine; installed with <code>pip install jinja2</code>
HTML & CSS	Basic templating used in the templates and static folders
Gemini API Key	Sign in to Google Cloud / Google AI Studio, accept the terms, click "Get API key", create the key, copy it and store it securely in the .env file

Assumptions and Constraints

- Cloud features need a valid Gemini API key and an internet connection.
- AI-generated suggestions depend on model output and may vary between runs.
- Third-party platform data is mocked or simulated in this version rather than fetched from live APIs.

3. Project Design Phase

System Architecture

PocketSmart AI has three main parts: the Frontend (HTML, CSS and Jinja2 templates), the Backend (FastAPI application) and the AI layer (Google Gemini 1.5 Flash Pro). The request flow is:

- Browser (HTML form + CSS)
- > FastAPI endpoints (main.py)
- > Planner logic (gemini_utils.py: prompts, budget formatting, image analysis)
- > Gemini 1.5 Flash Pro (cloud)
- > Recommendation cards shown on the page

Model Selection

Model	Used For	Benefits
Gemini 1.5 Flash Pro (via API)	Budget understanding, image analysis, structured suggestions, platform-specific product recommendations for Home, Party and Jewelry	Multimodal (text + image), fast responses, structured outputs, cloud inference

Module Design

Module	Responsibility
main.py	FastAPI app, endpoints, CORS, session handling, startup
gemini_utils.py	Prompt orchestration, budget formatting, domain segmentation, image analysis, fallback logic
routes/	API endpoints
services/	Recommendation logic and AI calls
models/	Input and output schemas
templates / static	Jinja2 HTML pages and CSS styling
.env	Gemini API key and configuration

API Endpoint Design

Endpoint	Purpose
/generate-home	Home interior recommendations (furniture, decor, lighting)
/generate-party	Catering, venue and decoration options for events
/generate-jewelry	Text + image input to return style-matched jewelry
/register, /login, /logout	User registration, authentication and session end
/token	Issues a JWT token after successful authentication
/session-info, /session-data	Session metadata and session-specific data for personalization
/recommendations-details	Detailed AI recommendations by budget, preferences and category
/history	User's past recommendation queries and results

AI Design

- Gemini receives a structured prompt for each planner (home, party or jewelry) containing the budget, preferences and platform list.
- For the Jewelry Planner the outfit image is sent together with the text prompt for colour and style analysis.
- Prompts are tuned for budget interpretation so that the suggested items stay within the entered budget.
- Errors are handled safely, and default recommendations are returned if the AI result is insufficient.

4. Project Planning Phase

Milestones

Milestone	Title	Activities
Milestone 1	Gemini AI Initialization	Create Google Cloud account, enable the Gemini API, create and copy the API key; validate connectivity with text-only and image + text prompts
Milestone 2	Core Functionalities Development	Initialize FastAPI, import libraries, load .env; build /generate-home, /generate-party, /generate-jewelry; add login, register and logout routes; session endpoints
Milestone 3	Backend – FastAPI Integration (main.py)	Define routes, modular architecture, /recommendations-details, CORS and static routing, /history, startup and main function
Milestone 4	UI Development	Build home, testimonials, register, login, dashboard, planner, recommendation and history pages with HTML, CSS and Jinja2
Milestone 5	Testing & Optimization	Test real-world budgets across all planners; refine prompts, session handling and UI/UX

Team Details

Role	Name
Team Leader	Gokulnathan J
Team Member	Giridharan Raja
Team Member	Kubenthiran T
Team Member	Santhosh V
Team Member	Thamaraiselvan S

Deliverables

- Source code (FastAPI application with all modules)
- requirements.txt and .env template
- HTML templates and CSS stylesheet
- Screenshots of every page and feature in action
- Phase-wise project report (this document)

Risks and Mitigation

Risk	Mitigation
Accuracy of AI-generated recommendations	Continuous testing and refinement of prompt structures
Budget not respected in suggestions	Budget-adherence checks and prompt tuning; validation of inputs
Platform data accuracy	Simulated platform data with platform-accuracy evaluation during testing
Insufficient or empty AI results	Fallback and default recommendations
Data privacy and security	API key kept in .env; JWT authentication; secure credential storage

5. Project Development Phase

Development Flow

- The user registers or logs in, opens the home page and chooses a planner (Home, Party or Jewelry).
- The user fills the form with budget and preferences; the Jewelry form also accepts an outfit image.
- Submitting the form sends a POST request to the matching FastAPI endpoint.
- The service layer builds a prompt and calls Gemini 1.5 Flash Pro, then links the result to platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO.
- The response is formatted into structured recommendations and saved to the user's history.
- Results appear as card-like layouts on the recommendations page.

Core Functions

Module	Description
Home Planner	Takes the budget, room types and quantities and recommends cost-effective furniture, decor and lighting balanced across function, style and price
Party Planner	Splits the budget across catering, decoration and entertainment, adds venue and stay options, and tailors the plan to the event type using conditions
Jewelry Planner	Takes budget, occasion, style and optional outfit image and returns matching jewelry options
Authentication & Session	/register, /login, /logout, /token (JWT), /session-info and /session-data for personalized access
History	/history stores and shows previous queries and results for review or re-use

Frontend Pages

- **Home Page** – landing page introducing the features with links to the planners.
- **Testimonials Page** – user reviews and success stories.
- **Register / Login Pages** – account creation and authentication.
- **User Dashboard** – recent recommendations, saved queries and personalized insights.
- **Planner and Recommendation Pages** – an input page and a results page for each of Home Interior, Party and Jewelry.
- **Recommendation History Page** – log of past queries and results.

Technology Stack

Layer	Technology
Language / Framework	Python 3.10+, FastAPI, Uvicorn (ASGI server)
Templating / Frontend	Jinja2, HTML, CSS, JavaScript
AI	Google Gemini 1.5 Flash Pro (via API key)
Security	JWT tokens, session management, CORS, .env configuration

How to Run

```

pip install -r requirements.txt
# add your Gemini API key to the .env file
uvicorn main:app --reload
# open http://127.0.0.1:8000

```

6. Project Testing Phase

Test Approach

The application was run locally with Uvicorn and tested end to end through the browser at <http://127.0.0.1:8000>. Each planner was tried with varied budgets and preferences to confirm that the correct module is called, the recommendations stay within budget and the results appear on the page.

Functional Test Cases

No.	Test	Steps	Expected Result
1	Application loads	Open http://127.0.0.1:8000	Home page with links to the planners is shown
2	Register and login	Create an account, then log in	User is authenticated and the dashboard opens
3	Home planner	Enter a budget, rooms and quantities	Cost-effective items from IKEA / Amazon within budget
4	Party planner	Enter budget, guests, event type, venue	Budget split across catering, decor and entertainment with Swiggy / Zomato / OYO options
5	Jewelry planner	Enter budget, occasion, style; upload an outfit image	Matching jewelry from Amazon / Flipkart
6	History	Open the history page	Past queries and results are listed
7	Logout	Click logout	Session ends and the login page opens
8	Error handling	Use an invalid API key or empty input	A helpful error message or fallback result is shown without crashing

Manual Testing Checklist

Input and Generation

- ☐ Home, Party and Jewelry forms accept budgets and preferences.
- ☐ Jewelry form accepts an optional outfit image.
- ☐ Each planner returns recommendations as cards.

Quality and Accounts

- ☐ Suggestions respect the entered budget and the right platforms.
- ☐ Register, login, logout and history work correctly.

Errors and Interface

- ☐ Missing or invalid API key produces a handled error message.
- ☐ Layout is responsive and readable on different screen sizes.

7. Project Documentation Phase

Technology Stack Summary

- Python 3.10+
- FastAPI and Uvicorn
- Jinja2, HTML and CSS
- Google Gemini 1.5 Flash Pro API
- JWT authentication and session management

AI Integration

PocketSmart AI uses Gemini 1.5 Flash Pro, a multimodal model, to interpret budgets, analyze outfit images, generate structured suggestions and recommend platform-specific products. All AI logic is kept in the utility file `gemini_utils.py`, which handles prompt orchestration, budget formatting, domain segmentation and image analysis, so the routes stay clean and each planner can be upgraded independently.

Security Considerations

- The Gemini API key is created in Google Cloud / AI Studio and stored in the `.env` file, never in the code.
- User credentials are stored securely and access to protected routes uses JWT tokens.
- CORS headers and session management control frontend communication.
- Errors are handled with clear messages instead of crashing the application.

Limitations

- Suggestions depend on AI output and should be checked before buying.
- Cloud features need an internet connection and a valid API key.
- Platform data is simulated in this version, so live prices and availability may differ.
- Currently run locally only.

Future Enhancements

- Live product and price integration with the Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO APIs.
- More planners, such as travel, wardrobe and gifting.
- Multilingual support and a mobile app.
- Wishlists, saved plans and shareable budget reports.

Conclusion

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across home interiors, party planning and jewelry selection. By combining FastAPI with Gemini 1.5 Flash Pro, the system processes budgets, preferences and even images to produce accurate, context-aware suggestions from trusted platforms. Its modular backend, secure session handling and responsive Jinja2 frontend make it efficient, scalable and user-centric, and it shows how generative AI can bring budget-friendly intelligent recommendations to everyday decisions.

8. Project Demonstration Phase

Suggested Demonstration Flow (3–5 minutes)

- **Introduction** – “PocketSmart AI is a smart budget and recommendation assistant for home, party and jewelry, powered by Gemini.”
- **Register and login** – create an account and open the dashboard.
- **Home planner** – enter a budget, rooms and quantities and show the recommendations.
- **Party planner** – enter budget, guests and event type and show the budget split.
- **Jewelry planner** – upload an outfit image and show the matching jewelry.
- **History** – open the recommendation history page.
- **Conclusion** – recap the flow: input -> FastAPI -> planner logic -> Gemini -> recommendations.

Team Summary

Team Member	Role
Gokulnathan J (Team Leader)	Team coordination and project lead
Giridharan Raja	Team member
Kubenthiran T	Team member
Santhosh V	Team member
Thamaraiselvan S	Team member

Closing Statement

PocketSmart AI demonstrates an end-to-end AI-integrated web application, from simple budget input, through the Gemini model, to personalized home, party and jewelry recommendations, all delivered as a modular and documented FastAPI project.